

Bulut Mimarilerinde Test Mühendisliği

Dönem Projesi: To-Do List Manager

Hazırlayan: Onur Burak Su - 170422505
Marmara Üniversitesi • Bilgisayar Mühendisliği • 17 Mayıs 2026

Problem & Çözüm

✗ Problem

Modern yazılımda test, dağıtım ve izleme süreçlerinin manuel ve hataya açık olması.

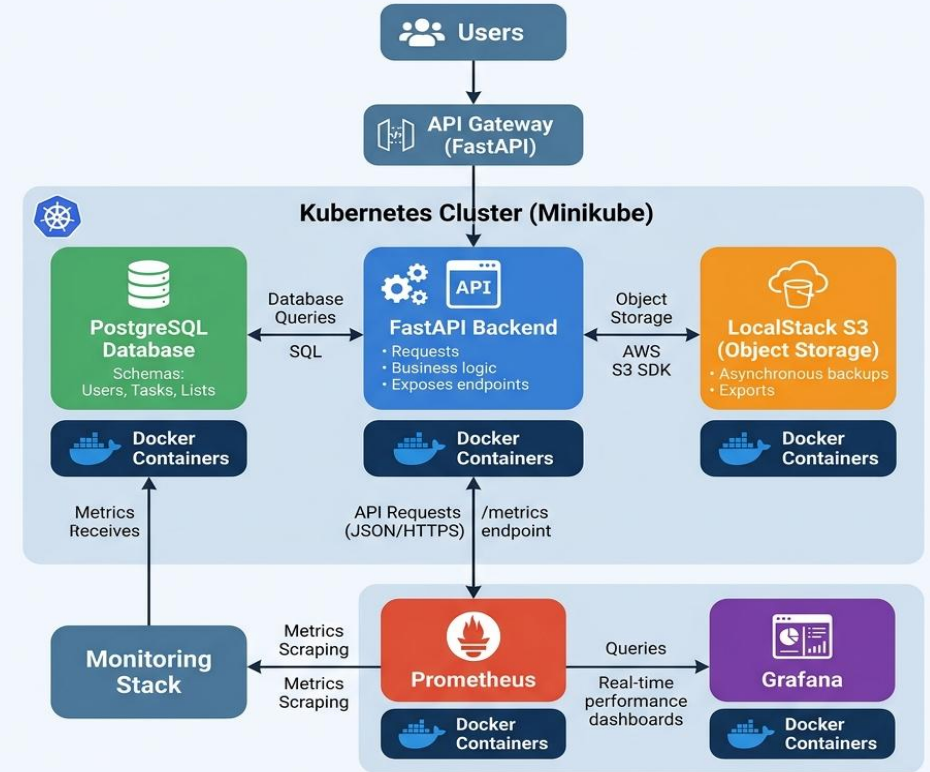
✓ Çözüm

Basit ama kapsayıcı bir uygulama üzerinden endüstri standardında test ve dağıtım altyapısı kurmak.

To-Do List Manager API

CRUD + Etiketleme + Yedekleme (S3)

TO-DO LIST MANAGER API ARCHITECTURE

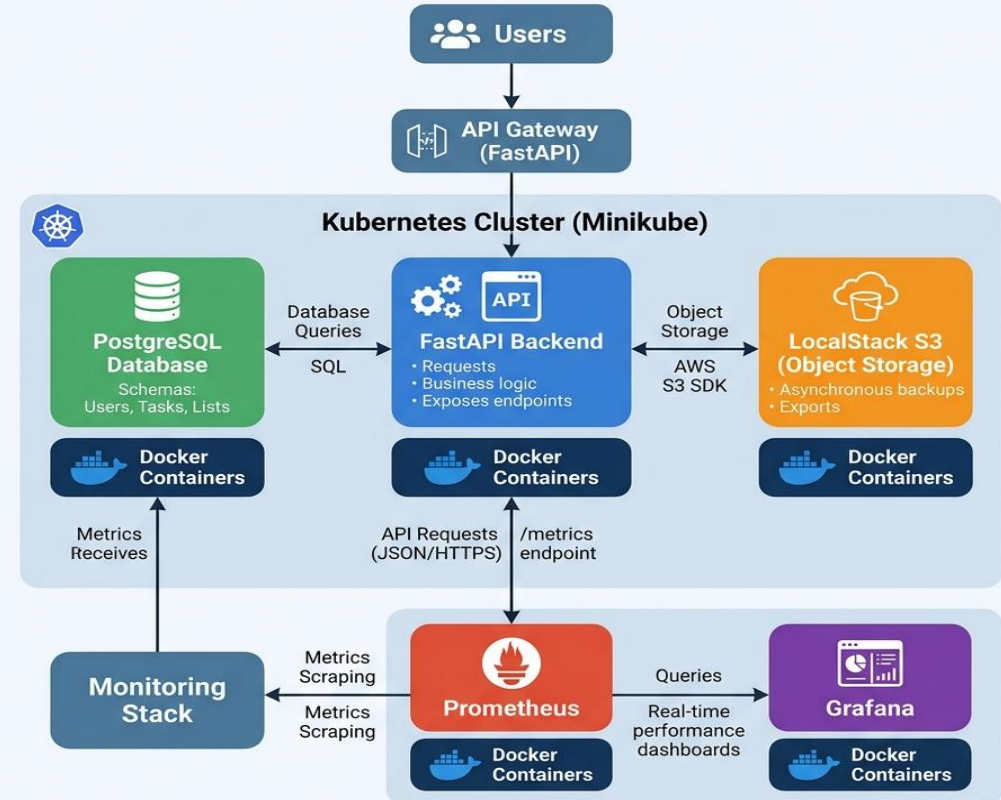


Flat Vector Style | Clean | Modern | Kubernetes (Minikube) Deployment

Mimari Diyagram

- ▶ Backend: Python FastAPI (3 Katmanlı)
- ▶ DB: SQLite (dev) + PostgreSQL (test)
- ▶ ORM: SQLAlchemy 2.0
- ▶ AWS: LocalStack S3 Yedekleme
- ▶ Container: Multi-Stage Dockerfile
- ▶ Orkestrasyon: Kubernetes (Minikube)
- ▶ İzleme: Prometheus + Grafana

TO-DO LIST MANAGER API ARCHITECTURE



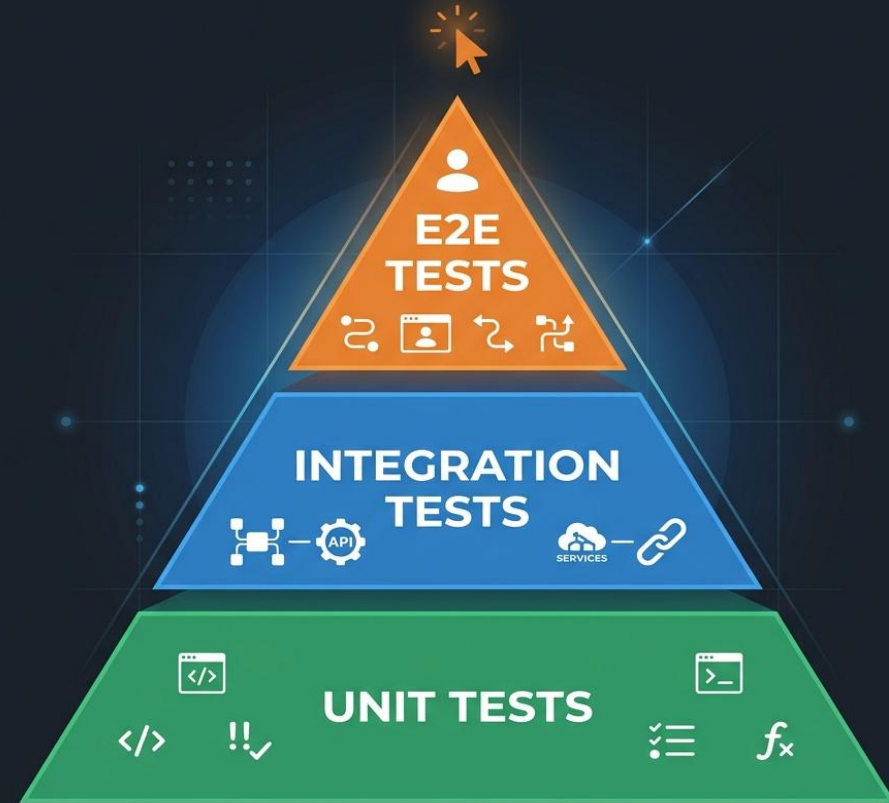
Flat Vector Style | Clean | Modern | Kubernetes (Minikube) Deployment

Test Stratejisi

Test Piramidi yaklaşımı uygulandı.

- ◆ Unit Tests: Pytest + Factory Boy + Faker
- ◆ Integration: Testcontainers (PostgreSQL)
- ◆ E2E: Playwright (tarayıcı otomasyonu)
- ◆ API: Postman + Newman (11 assertion)
- ◆ Performans: k6 yük testi

Hedef: %70 → Sonuç: %83 Coverage ✓



Birim & Entegrasyon Testleri

Birim Testler (Unit)

- Service ve Route katmanları izole test edildi
- Factory Boy ile dinamik test verisi üretildi
- Her testte DB sıfırlanarak izolasyon sağlandı

Entegrasyon Testleri

- Testcontainers ile gerçek PostgreSQL ayağa kalkar
- ORM CRUD işlemleri gerçek DB üzerinde doğrulanır
- Docker kapalıysa pytest.skip ile atlama yapılır

Kapsam Raporu (Coverage)

main.py	95%	✓
task_routes.py	96%	✓
task_service.py	81%	✓
task.py (model)	87%	✓
schemas/task.py	100%	✓

TOPLAM:	%83	✓
---------	-----	---

E2E & API Testleri

E2E Testleri (Playwright)

- Statik HTML arayüz /ui üzerinden sunuldu
- Görev Ekleme → DOM'a düşme doğrulandı
- Tamamla butonu → Durum değişimi test edildi
- Sil butonu → DOM'dan kalkma doğrulandı

API Testleri (Postman + Newman)

- 6 API isteği ve 11 assertion içeren koleksiyon
- Dinamik veri aktarımı (task_id değişkeni)
- npm run test:api ile CI'da otomatik koşulur

CI/CD Pipeline (GitHub Actions)

MODERN CI/CD PIPELINE FLOW



1 ☐ Lint (flake8) → 2 ☐ Pytest → 3 ☐ Docker Build → 4 ☐ Deploy (sim.) → 5 ☐ Smoke Test

Her push/PR ile otomatik tetiklenir • Tüm adımlar geçmeden merge yapılamaz

Container & Kubernetes

Docker

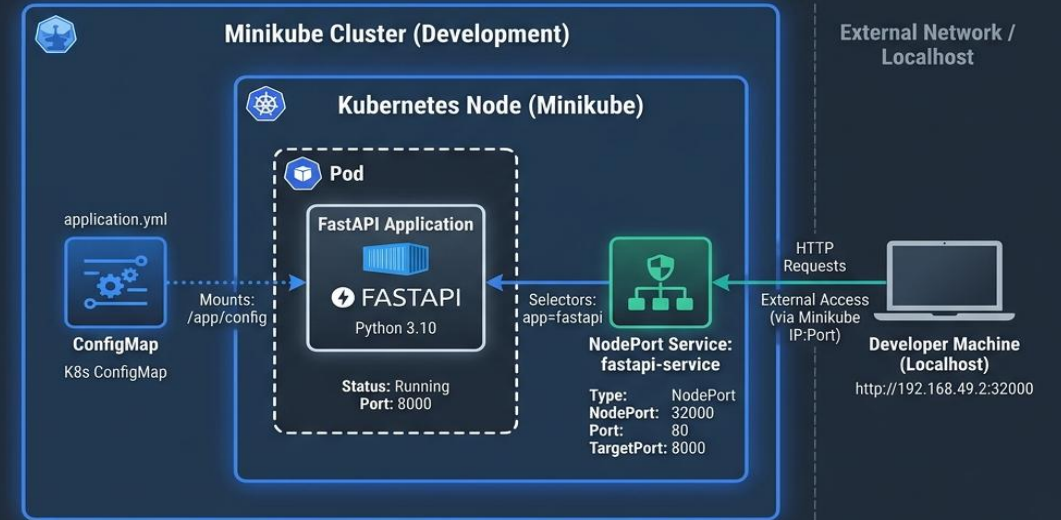
- Multi-stage build (güvenlik + boyut)
- docker-compose ile tüm servisler ayağa kalkar

Kubernetes (Minikube)

- ConfigMap: Ortam değişkenleri
- Deployment: Liveness/Readiness probes
- Service: NodePort ile dış erişim

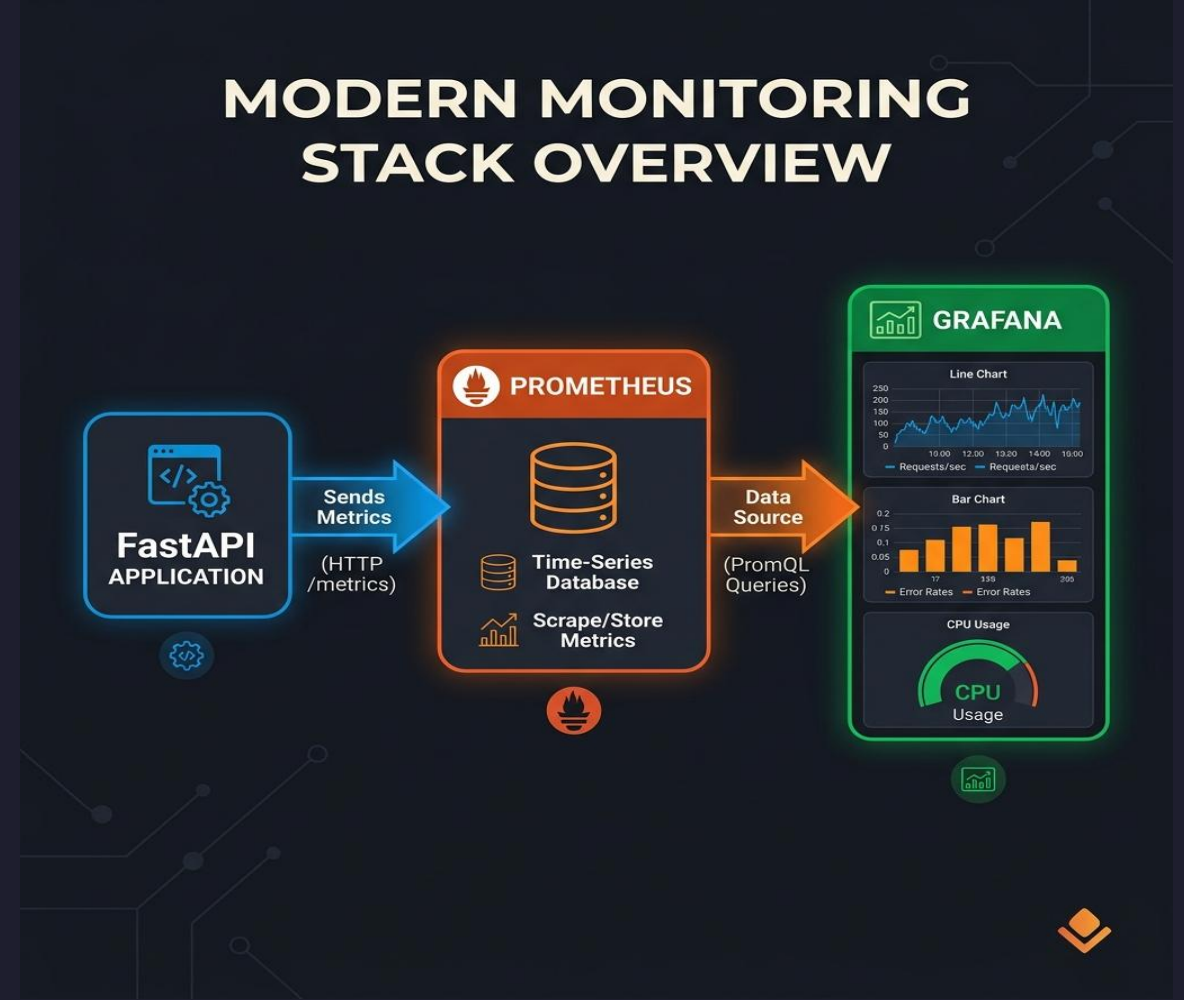
Kubernetes Deployment Architecture: FastAPI on Minikube

Clean, Modern Vector Diagram | Presentation Slide Style | Professional Blue & Teal Palette



Monitoring & Observability

- ▶ prometheus-fastapi-instrumentator ile /metrics endpoint'i aktif edildi
- ▶ Prometheus ile periyodik veri toplama
- ▶ Grafana Dashboard (4 Panel):
 - Throughput (İstek/sn)
 - Latency (Yanıt süresi)
 - Error Rate (Hata oranı)
 - Toplam İstek Sayısı



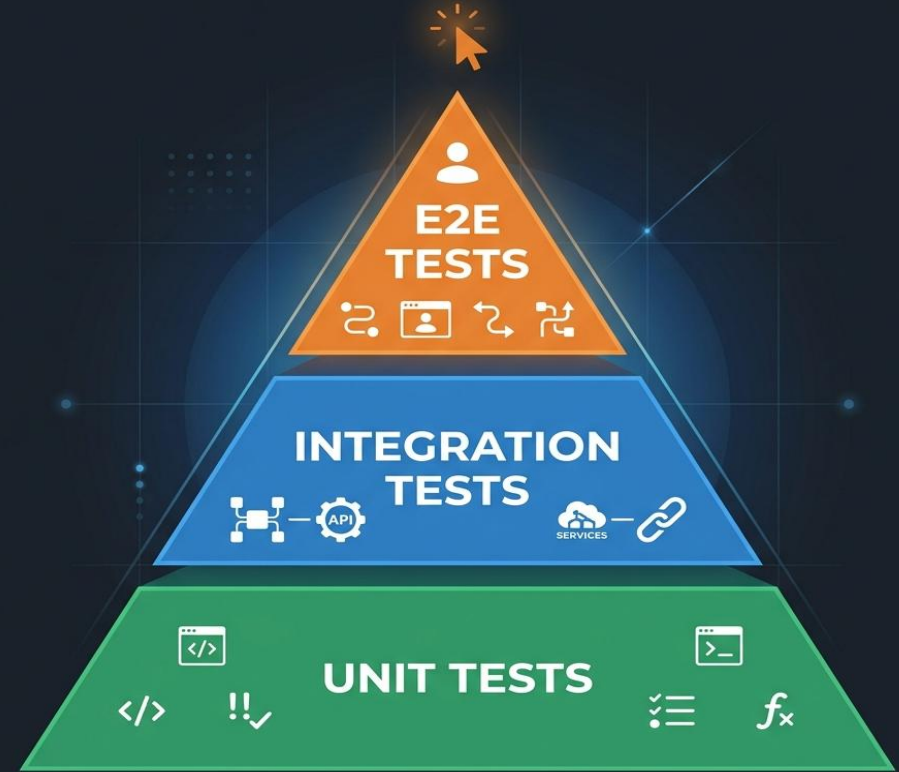
Performans & Sayılarla Proje

k6 Yük Testi Senaryosu

- 10sn warm-up → 20 VU → 30sn tepe → cool-down
- Hedef p95 Latency: < 500ms
- Hedef Hata Oranı: < %1

Sayılarla Proje Özeti

◆ REST API Endpoint:	10
◆ Kod Kapsamı (Coverage):	%83
◆ Postman Assertions:	11
◆ CI/CD Pipeline Adımı:	5
◆ Grafana Panel Sayısı:	4
◆ K8s Manifest Dosyası:	3
◆ Entitv (Task + Tag):	2



Öğrendiklerim & Zorluklar

Karşılaşılan Zorluklar:

1 ☐ Test İzolasyonu

Testler arası durum sızıntısı → `conftest.py` içinde her test öncesi DB sıfırlama uygulandı.

2 ☐ Ortam Değişkenleri Güvenliği

`.env` dosyası yerine K8s ConfigMap kullanıldı.

3 ☐ Docker + Testcontainers

Windows'ta Docker erişim sorunları → `try-except` ile graceful fallback sağlandı.

Öğrenilen Dersler:

✓ Kod yazmak işin sadece yarısı;
test, dağıtım ve izleme altyapısı
profesyonel yazılımın can damarıdır.

✓ CI/CD pipeline sayesinde hatalar
üretime çıkmadan yakalanır.

✓ Kubernetes ve Docker ile ortamlar
arası tutarlılık garantilenir.

Teşekkürler! — Soru & Cevap